AI Project Evaluator — Viva Module API Documentation

Base URL: `http://127.0.0.1:8000/api/viva/`
Authentication: All endpoints require a JWT Bearer token in the `Authorization` header.
Content-Type: `application/json` unless noted as `multipart/form-data`.

Authentication

All requests must include:

```
Authorization: Bearer <access_token>
```

Login

```
POST http://127.0.0.1:8000/api/auth/login/
```

Request:

```json
{
    "email": "examiner@test.com",
    "password": "your_password"
}
```

Response:

```json
{
    "access": "<access_token>",
    "refresh": "<refresh_token>"
}
```

Access token expires in **60 minutes**. Use the refresh token to get a new one.

Refresh Token

```
POST http://127.0.0.1:8000/api/auth/token/refresh/
```

Request:

```json
{
    "refresh": "<refresh_token>"
}
```

User Roles

Role	Can Do
`examiner`	Create projects, rubrics, sessions, view reports, submit final scores
`student`	Upload reports, answer viva questions

---

1. Students

List All Students

Used by the examiner when creating a session — to pick which student to assign.

```
GET /api/viva/students/
```

Response:

```json
[
    {
        "id": "74d9650f-60c9-4822-a88f-597c907b315d",
        "full_name": "Test Student",
        "email": "student@test.com",
        "registration_number": "REG001",
        "degree_program": "BSc Computer Science",
        "academic_year": 4,
        "batch": "2021"
    }
]
```

---

2. Projects

List All Projects

Examiner's project dashboard.

```
GET /api/viva/projects/
```

Response: Array of project objects (same format as Project Detail below).

Get Project Detail

Returns full project with nested rubric — categories, criteria, and hints.

```
GET /api/viva/projects/<project_id>/
```

Response:

```
json
```

```
{
    "id": "e606c8c0-c4d0-4775-9404-6925b2895988",
    "project_name": "Plant Disease Detection using CNN",
    "description": "A deep learning project...",
    "is_group_project": false,
    "submission_deadline": null,
    "academic_year": "2024/2025",
    "status": "draft",
    "created_at": "2026-05-09T03:05:54.746736Z",
    "rubric_categories": [
        {
            "id": "f46e707e-8398-44e9-9a57-9d0c7c73bbcc",
            "category_name": "Problem Formulation",
            "weight_percentage": "60.00",
            "description": "...",
            "criteria": [
                {
                    "id": "b68dbf0b-191a-4d5c-85a0-eb21d374397d",
                    "criteria_name": "Problem Identification",
                    "max_score": "10.00",
                    "weight_in_category": null,
                    "description": "...",
                    "questions_to_ask": 3,
                    "question_hints": [
                        {
                            "id": "uuid",
                            "hint_text": "Ask about the specific problem...",
                            "order": 1
                        }
                    ]
                }
            ],
            "weight_warning": null
        }
    ]
}
```

Create Project with Rubric

Creates a project and full rubric in a single call.

```
POST /api/viva/projects/
```

Request:

```json
{
    "project_name": "Plant Disease Detection using CNN",
    "description": "A deep learning based system...",
    "is_group_project": false,
    "academic_year": "2024/2025",
    "submission_deadline": "2026-06-01T00:00:00Z",
    "rubric_categories": [
        {
            "category_name": "Problem Formulation",
            "weight_percentage": 50.00,
            "description": "Evaluates problem definition",
            "criteria": [
                {
                    "criteria_name": "Problem Identification",
                    "max_score": 10.00,
                    "weight_in_category": 50.00,
                    "description": "How well the student identifies the problem",
                    "questions_to_ask": 3,
                    "question_hints": [
                        {
                            "hint_text": "Ask about specific bottlenecks identified"
                            "order": 1
                        }
                    ]
                }
            ]
        }
    ]
}
```

Response: Full project detail object (same as Get Project Detail) plus optional warnings:

```json
```

```
{
    "id": "...",
    "project_name": "...",
    "rubric_categories": [...],
    "warnings": []
}
```

> `warnings` is an array of strings. If category weights don't add up to 100%, a warning is included but the project is still saved.

Add Rubric Category to Existing Project

```
POST /api/viva/projects/<project_id>/categories/
```

Request:

```json
{
    "category_name": "Methodology",
    "weight_percentage": 30.00,
    "description": "Evaluates research methodology"
}
```

Response: The created category object.

Update Rubric Category

```
PATCH /api/viva/categories/<category_id>/
```

Request (all fields optional):

```json
{
    "category_name": "Updated Name",
    "weight_percentage": 40.00,
    "description": "Updated description"
}
```

Response: Updated category fields.

Add Criterion to Category

```
POST /api/viva/categories/<category_id>/criteria/
```

Request:

```json
{
    "criteria_name": "Technical Approach",
    "max_score": 10.00,
    "weight_in_category": 50.00,
    "description": "Evaluates technical decisions made",
    "questions_to_ask": 3,
    "question_hints": [
        {
            "hint_text": "Ask about architecture choices",
            "order": 1
        }
    ]
}
```

Response: The created criterion object.

Update Criterion

```
PATCH /api/viva/criteria/<criteria_id>/
```

Request (all fields optional):

```json
{
    "criteria_name": "Updated Name",
    "max_score": 15.00,
    "description": "Updated description",
    "questions_to_ask": 4
}
```

Response: Updated criteria fields.

Add Hint to Criterion

```
POST /api/viva/criteria/<criteria_id>/hints/
```

Request:

```json
{
    "hint_text": "Ask about the specific dataset used",
    "order": 1
}
```

Response:

```json
{
    "id": "uuid",
    "hint_text": "Ask about the specific dataset used",
    "order": 1
}
```

Delete a Hint

```
DELETE /api/viva/hints/<hint_id>/
```

Response:

```json
{
    "message": "Hint deleted successfully."
}
```

---

3. Rubric Upload (PDF/DOCX)

Alternative to manually creating a rubric — examiner uploads a rubric document and the AI extracts the structure.

Step 1 — Upload and Preview

```
POST /api/viva/rubric/upload-preview/
Content-Type: multipart/form-data
```

Form fields:

Field	Type	Required
`rubric_file`	File (PDF or DOCX)	Yes

Response:

```json
{
    "message": "Rubric extracted successfully. Review and confirm to save.",
    "preview": {
        "project_name": "...",
        "project_description": "...",
        "is_group_project": false,
        "academic_year": "2024/2025",
        "rubric_categories": [...],
        "extraction_notes": "Assumptions made by AI..."
    }
}
```

The examiner reviews and edits the `preview` object on the frontend before confirming.

Step 2 — Confirm and Save

```
POST /api/viva/rubric/confirm-save/
```

Request: Send the (optionally edited) `preview` object from Step 1 as the request body.

Response:

```json
{
    "message": "Rubric saved successfully.",
    "project": { ...full project detail object... }
}
```

---

4. Submissions

Upload Student Report

Student uploads their PDF or DOCX report. Text is extracted immediately.

```
POST /api/viva/submissions/upload/
Content-Type: multipart/form-data
```

Behaviour by role:

- **Student logged in** → student is auto-detected. Only `project` and `report_file` needed.
- **Examiner logged in** → must also pass `student` UUID.

Form fields:

Field	Type	Required
`project`	UUID	Yes
`report_file`	File (PDF or DOCX)	Yes
`student`	UUID	Only if examiner is uploading
`group`	UUID	Optional

Response:

```json
```

```
{
    "message": "Report uploaded and processed successfully.",
    "submission_id": "e0aa124f-9d48-49ba-8362-251db24edf6d",
    "status": "ready",
    "characters_extracted": 37520
}
```

Check Submission Status

```
GET /api/viva/submissions/<submission_id>/status/
```

Response:

```json
{
    "submission": "uuid",
    "status": "ready",
    "error_message": null,
    "processed_at": "2026-05-12T10:00:00Z"
}
```

Status values:

Value	Meaning
`pending`	File uploaded, not yet processed
`processing`	Text extraction in progress
`ready`	Text extracted, session can start
`failed`	Extraction failed, check `error_message`

> Always check status is `ready` before allowing the viva session to start.

---

5. Evaluation Sessions

Create Session

Examiner links a project, student, and submission to create a session.

```
POST /api/viva/sessions/create/
```

Request:

```json
{
    "project": "e606c8c0-c4d0-4775-9404-6925b2895988",
    "student": "74d9650f-60c9-4822-a88f-597c907b315d",
    "submission": "e0aa124f-9d48-49ba-8362-251db24edf6d",
    "scheduled_start": "2026-05-12T10:00:00Z",
    "scheduled_end": "2026-05-12T11:00:00Z",
    "location_room": "Lab 301"
}
```

> Provide either `student` or `group`, not both.

Response:

```json
{
    "id": "23690262-a461-4e18-96b7-6fc5f381b940",
    "project": "uuid",
    "student": "uuid",
    "group": null,
    "submission": "uuid",
    "scheduled_start": "2026-05-12T10:00:00Z",
    "scheduled_end": "2026-05-12T11:00:00Z",
    "actual_start": null,
    "location_room": "Lab 301",
    "status": "scheduled"
}
```

List Sessions for a Project

```
GET /api/viva/projects/<project_id>/sessions/
```

Response: Array of session objects (same format as Create Session response).

Get Session Status

Frontend polls this to track session progress.

```
GET /api/viva/sessions/<session_id>/status/
```

Response:

```json
{
    "session_id": "uuid",
    "status": "in_progress",
    "scheduled_start": "2026-05-12T10:00:00Z",
    "scheduled_end": "2026-05-12T11:00:00Z",
    "actual_start": "2026-05-12T10:05:00Z",
    "total_questions_asked": 4,
    "total_answers_submitted": 4
}
```

Session status values:

Value	Meaning
`scheduled`	Created, not yet started
`in_progress`	Viva is actively running
`completed`	All criteria covered, viva done

---

6. Viva Flow

This is the core loop. Once a session is created and the submission is `ready`, the viva proceeds as follows:

```
Start Session → Get Question → Submit Answer → Get Next Question → ... → Session Complete → Get Report
```

Start Session

Generates and returns the first question.

```
POST /api/viva/sessions/start/
```

Request:

```json
{
    "session_id": "23690262-a461-4e18-96b7-6fc5f381b940"
}
```

Response:

```json
{
    "message": "Session started.",
    "session_id": "uuid",
    "question_id": "uuid",
    "question_text": "How did you identify the core problem in your project?",
    "blooms_level": "Apply",
    "difficulty": "medium",
    "criterion": "Problem Identification",
    "question_number": 1
}
```

> Save `question_id` — you need it to submit the answer.

Submit Answer

Student submits their answer. Returns score feedback and the next question.

```
POST /api/viva/sessions/<session_id>/answer/
```

Request:

```json
{
    "question_id": "uuid-of-current-question",
    "answer_text": "The student's spoken answer transcribed here"
}
```

Response — session still in progress:

```json
{
    "answer_saved": true,
    "score": 7.5,
    "reasoning": "The student demonstrated understanding of CNN architecture...",
    "strengths": "Clear explanation of convolutional layers",
    "gaps": "Did not mention pooling layers or regularization",
    "session_complete": false,
    "next_question": {
        "question_id": "uuid",
        "question_text": "Can you explain why you chose this specific CNN architectu
        "blooms_level": "Analyze",
        "difficulty": "medium",
        "criterion": "Technical Approach",
        "question_number": 2
    }
}
```

Response — session complete:

```json
{
    "answer_saved": true,
    "score": 8.0,
    "reasoning": "...",
    "strengths": "...",
    "gaps": "...",
    "session_complete": true,
    "message": "All criteria have been covered. Session is complete."
}
```

> When `session_complete` is `true`, redirect the examiner to the report page.

Score range: 0–10
Difficulty values: `easy`, `medium`, `hard`
Bloom's levels: `Remember`, `Understand`, `Apply`, `Analyze`, `Evaluate`, `Create`

Get XAI Report

Available only after `session_complete` is `true`. Returns AI scores and full explanation for the examiner.

```
GET /api/viva/sessions/<session_id>/report/
```

Response:

```json
```

```
{
    "session_id": "uuid",
    "overall_score": 7.23,
    "per_criterion_scores": {
        "Problem Identification": 7.75,
        "Research Gap": 7.5,
        "Technical Approach": 6.6
    },
    "xai_report": {
        "overall_summary": "The student demonstrated solid understanding...",
        "strengths": "Strong grasp of CNN fundamentals...",
        "gaps": "Limited understanding of validation strategies...",
        "examiner_recommendation": "Recommend a pass with distinction..."
    },
    "transcript": [
        {
            "question": "How did you identify the core problem?",
            "blooms_level": "Apply",
            "answers": [
                {
                    "answer": "I identified the problem by...",
                    "score": 7.5,
                    "reasoning": "...",
                    "next_difficulty": "same"
                }
            ]
        }
    ]
}
```

---

7. Final Scores

After reviewing the XAI report, the examiner approves or adjusts the AI scores and submits final grades.

```
POST /api/viva/sessions/<session_id>/final-scores/
```

> Must be called with an **examiner** token. Students cannot call this endpoint.

Request:

```json
{
    "scores": [
        {
            "criteria_id": "b68dbf0b-191a-4d5c-85a0-eb21d374397d",
            "examiner_final_score": 8.0,
            "examiner_note": "Strong problem identification with good justification"
        },
        {
            "criteria_id": "79ca183b-81b1-4abf-b1eb-6234412911ea",
            "examiner_final_score": 7.5,
            "examiner_note": "Research gap well articulated"
        },
        {
            "criteria_id": "331e2fcc-7625-43ec-8ee7-6b943a3729ca",
            "examiner_final_score": 6.5,
            "examiner_note": "Technical approach needs more depth"
        }
    ],
    "overall_feedback": "Student showed solid grasp of project concepts overall.",
    "grade": "B+"
}
```

Response:

```json
{
    "message": "Final scores submitted successfully.",
    "session_id": "uuid",
    "grade": "B+",
    "total_final_score": 22.0,
    "total_ai_score": 21.85,
    "scores": [
        {
            "criteria": "Problem Identification",
            "ai_recommended_score": 7.75,
            "examiner_final_score": 8.0,
            "examiner_note": "Strong problem identification..."
        }
    ]
}
```

> `ai_recommended_score` shows what the AI suggested. `examiner_final_score` is what was actually saved. The examiner can override the AI score for any criterion.

---

8. Error Responses

All endpoints return errors in this format:

```json
{
    "error": "Description of what went wrong."
}
```

Common HTTP status codes:

Code	Meaning
`200`	Success
`201`	Created successfully
`400`	Bad request — missing or invalid fields
`401`	Unauthorized — missing or expired token
`403`	Forbidden — wrong role (e.g. student trying examiner action)
`404`	Resource not found
`500`	Server error

---

9. Complete Frontend Flow

Examiner Flow

```
1. Login → get token
2. GET /students/ → pick student for session
3. POST /projects/ → create project with rubric
   OR POST /rubric/upload-preview/ + POST /rubric/confirm-save/
4. POST /submissions/upload/ → upload student report (if examiner is uploading)
```

```
5. GET /submissions/<id>/status/ → wait for "ready"
6. POST /sessions/create/ → create session
7. POST /sessions/start/ → start viva, get first question
8. Display questions, collect answers, POST /sessions/<id>/answer/ in loop
9. When session_complete: true → GET /sessions/<id>/report/
10. Review XAI report → POST /sessions/<id>/final-scores/
```

Student Flow

```
1. Login → get token
2. POST /submissions/upload/ → upload report (student auto-detected)
3. GET /submissions/<id>/status/ → confirm "ready"
4. Wait for examiner to start the session
5. POST /sessions/start/ → first question appears
6. POST /sessions/<id>/answer/ in loop until session_complete: true
```

---

10. Notes for Frontend

- Always store `question_id` from each question response — it is required to submit the answer.
- Poll `GET /sessions/<id>/status/` to track live session progress.
- The `session_complete` flag in the answer response is the signal to stop the answer loop and redirect to the report.
- Criteria IDs needed for final score submission can be found in `GET /projects/<id>/` under `rubric_categories → criteria → id`.
- Token expires in 60 minutes — implement refresh token logic using `POST /api/auth/token/refresh/`.
- All IDs are UUIDs.